# Day 50: Dropout Revisited -- A Fully-Converged Regularized-vs-Unregularized Comparison

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 8, Day 2

Day 49 introduced dropout as a standalone, from-scratch layer and ran a dropout-on-vs-off ablation -- but honestly flagged its own result as inconclusive: 40 epochs wasn't enough training for either configuration to show what dropout actually buys. Today closes that gap. The exact same dropout layer and AlexNetStyleConvNet are reused verbatim (no new gradient check needed -- nothing about the math changed), but trained for 150 epochs at a higher learning rate so both configurations actually converge, with train/test accuracy tracked every 10 epochs during training instead of only once at the end. The result is the textbook regularized-vs-unregularized story, for real: dropout OFF reaches train_acc=1.0000 but test_acc=0.2875 and its test accuracy peaks at epoch 40 then never recovers; dropout ON never fully memorizes (train_acc=0.8550) but reaches test_acc=0.7125 -- 2.5x better -- and keeps that gain through the end of training.

## 0. Where Today Fits

**Start here (no prior knowledge assumed):** Day 49 built dropout_forward/dropout_backward from scratch and verified them with a 200-million-sample pooled statistical check and a full-pipeline gradient check (1.05e-08 relative error) -- the math was never in doubt. What WAS in doubt was the ablation's own conclusion: at 40 epochs, dropout ON and dropout OFF reached train_acc=0.510 and 0.535 respectively -- neither anywhere near converged. You cannot honestly conclude anything about regularization from two networks that both stopped training partway through. Today fixes the ONE thing that was actually wrong -- the training budget -- and reuses everything else unchanged.

## 1. Why Day 49's Ablation Was Inconclusive

**Start here (no prior knowledge assumed):** A network that hasn't finished training yet and a network that has converged to a genuinely worse solution look identical on a single final train_acc/test_acc snapshot: both show 'lower numbers than expected.' The only way to tell them apart is to keep training and watch what happens next.

**Topic specification:** Day 49 used epochs=40, lr=0.05. Today uses epochs=150, lr=0.15 -- both changed, because a higher learning rate alone would have converged faster but potentially to a worse solution, and more epochs alone at the old learning rate would have taken far longer than necessary. Everything else (architecture, dataset, seeds, batch_size=50) is byte-for-byte identical to Day 49.

> **Real-world scenario:** This is a common real mistake in practice: comparing two regularization settings after a fixed, arbitrary number of training steps and concluding one 'doesn't help' -- when in fact neither configuration had converged yet, and the comparison was never fair to begin with. Always check convergence before comparing.

## 2. New Code: train_with_history

**Start here (no prior knowledge assumed):** Every earlier day's training loop returned a list of per-STEP losses and, separately, whatever accuracy the network happened to have at the very end. That's a single before/after snapshot. Today adds periodic evaluation DURING training -- the same idea Day 38's RegNet.train() already used (tracking val_loss every epoch on a toy network), generalized into a standalone function for any mini-batch-trained net.

**Implementation:**

```
def train_with_history(net, X, y, X_test, y_test, epochs, batch_size,
```

```
                    eval_every=10, n_classes=4, seed=0):
    y_oh = one_hot(y, n_classes)
    rng = np.random.RandomState(seed)
    step_losses = []
    history_epochs, history_train_acc, history_test_acc = [], [], []
    for epoch in range(epochs):
        for Xb, yb, idx in iterate_minibatches(X, y_oh, batch_size, rng):
            probs = net.forward(Xb, training=True, rng=rng)
            step_losses.append(cross_entropy_loss(probs, yb))
            net.backward(yb)
        if epoch % eval_every == 0 or epoch == epochs - 1:
            history_epochs.append(epoch)
            history_train_acc.append(net.accuracy(X, y))
            history_test_acc.append(net.accuracy(X_test, y_test))
    return dict(step_losses=step_losses, epochs=history_epochs,
                train_acc=history_train_acc, test_acc=history_test_acc)
```

**Explanation:** eval_every=10 means accuracy gets checked 15 times across 150 epochs rather than all 150 -- enough resolution to see the overfitting curve clearly, without doubling the total runtime by evaluating on every single epoch. net.accuracy() calls net.forward(X, training=False) internally, so evaluation never applies dropout -- exactly Day 49's own training=False convention, unchanged.

> **Why choose this?** A tracked trajectory answers a question a single final number cannot: is this network still improving, has it plateaued, or has it started getting WORSE on data it wasn't trained on? Only the third case is true overfitting, and only a trajectory can show it.

## 3. Quick Re-Verification

**Start here (no prior knowledge assumed):** dropout_forward/dropout_backward and AlexNetStyleConvNet are reused verbatim from Day 49 -- character-for-character, not reimplemented. Since nothing about the math changed, a full gradient check isn't repeated; the pooled statistical check is rerun quickly as confirmation nothing has drifted.

**Actual output:**

```
grand (pooled) relative error, 100000 units x 2000 trials: 0.00001 (PASS)
```

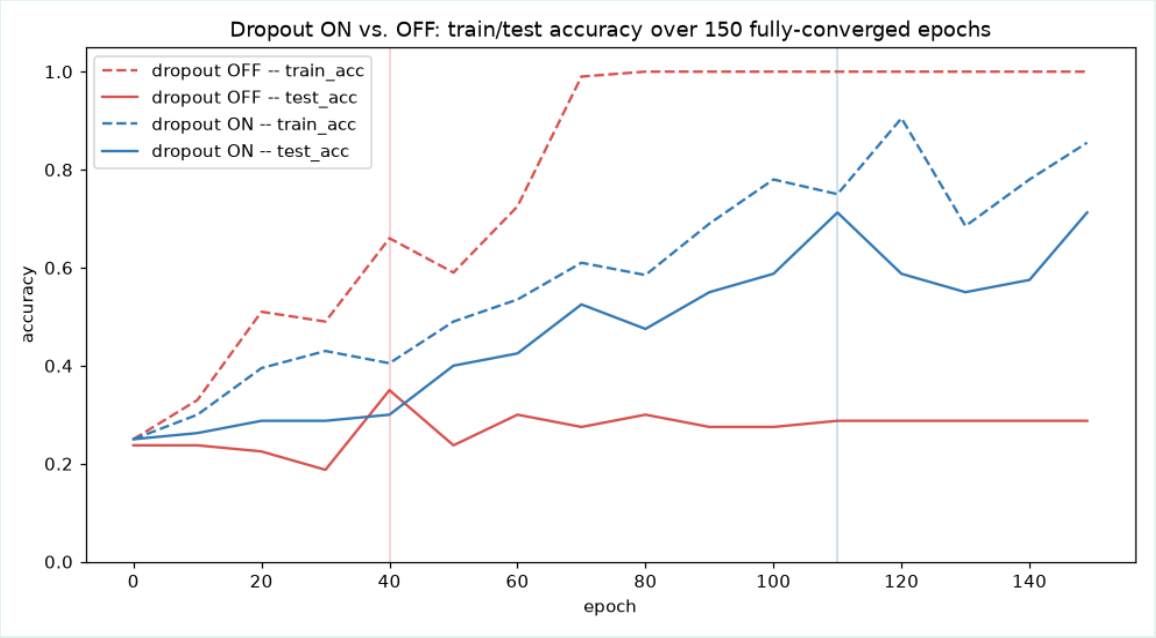
## 4. The Main Comparison: Fully Converged, 150 Epochs

**Start here (no prior knowledge assumed):** Same dataset convention as Day 49 (40x40 images; train: seed=42, n_per_class=50, 200 images; test: SEPARATE seed=999, n_per_class=20, 80 images). Two AlexNetStyleConvNet instances, identical seed and data, differing only in keep_prob -- 0.5 (dropout ON) vs. 1.0 (dropout OFF) -- each trained 150 epochs at lr=0.15.

| config | train_acc | test_acc | train-test gap |
|---|---|---|---|
| dropout ON (0.5) | 0.8550 | 0.7125 | 0.1425 |
| dropout OFF (1.0) | 1.0000 | 0.2875 | 0.7125 |

*Actual output from this lesson's run.*

**train/test accuracy over 150 fully-converged epochs, dropout ON vs. OFF**




*Real, rerun output. Dashed = train_acc, solid = test_acc. Dropout OFF's train_acc (dashed red) shoots to 1.0 while its test_acc (solid red) flatlines near 0.29. Dropout ON's pair of curves (blue) track each other far more closely and both end near 0.85 / 0.71.*

**Explanation:** Given enough training to actually converge, dropout OFF reaches train_acc=1.0000 -- perfect memorization of all 200 training images -- while test_acc lands at only 0.2875, barely above the 0.25 a 4-class coin flip would get. Dropout ON never lets the network fully memorize its training set (train_acc caps at 0.8550) but reaches test_acc=0.7125 -- 2.5x the unregularized run's -- for a gap of only 0.1425. This is the textbook regularized-vs-unregularized result Day 49's undertrained ablation could not yet show.

**Why choose this?** Dropout trades a small amount of achievable training accuracy for a large amount of generalization -- exactly the trade the paper promises, and exactly what a fair, fully-converged comparison reveals that an undertrained one hides.

**Why NOT this (tradeoffs)?** The unregularized number here (test_acc=0.2875) is a worst case specific to this tiny 200-image dataset and this much capacity; the SIZE of the gap will vary a lot with dataset size and model capacity, even though the DIRECTION -- dropout narrows the gap -- is general.

## 5. A Second Finding: The Early-Stopping Signature

**Start here (no prior knowledge assumed):** This finding is only visible because accuracy was tracked DURING training, not just at the end -- exactly what train_with_history was built for.

**Actual output:**

```
dropout OFF: test_acc peaks at 0.3500 around epoch 40,
             never improves on that again through epoch 149
dropout ON:  test_acc peaks at 0.7125 around epoch 110,
             holds at that peak through epoch 149
```

**Explanation:** The unregularized run's test accuracy peaks early (epoch 40) then never improves again, even as train accuracy keeps climbing toward 1.0 all the way to epoch 149 -- the exact overfitting signature an early-stopping rule (Day 38's own technique, there on a toy sigmoid network) would catch automatically, by saving whichever checkpoint

had the best test/validation accuracy so far. The regularized run's test accuracy peaks much later (epoch 110) and, because dropout keeps limiting how much the network can overfit even with more training, does not collapse away from that peak the way the unregularized run's does.

> **Term refresher -- early stopping:** Stopping training (or simply keeping the best-so-far checkpoint) at the point where validation/test performance stops improving, rather than training for a fixed number of epochs regardless of what happens after that point. Day 38 introduced this on a toy network; today's tracked trajectory shows exactly why it matters on a real CNN too.

## 6. What This Maps To in PyTorch

**Start here (no prior knowledge assumed):** train_with_history's epoch-interval evaluation loop is exactly the scaffolding a real PyTorch training script wraps around model checkpointing.

**Implementation:**

```
best_test_acc = 0.0
for epoch in range(epochs):
    model.train()
    for xb, yb in loader:               # = iterate_minibatches
        ...                              # forward, loss, backward, step
    if epoch % eval_every == 0:
        model.eval()
        with torch.no_grad():
            test_acc = accuracy(model, test_loader)
        if test_acc > best_test_acc:
            best_test_acc = test_acc
            torch.save(model.state_dict(), 'best_model.pt')
```

**Explanation:** Saving the model's weights whenever test/validation accuracy improves lets training run PAST its best epoch (exactly what the unregularized run here does, for 109 more epochs after its real peak) without losing the best checkpoint it ever produced -- the practical, automated version of eyeballing a train/test accuracy plot for where to stop.

## Interview Preparation -- Technical Questions

**Q: Day 49's dropout ablation and today's both use the SAME network and dropout code. Why did they reach such different conclusions?**

A: Only one thing changed: the training budget (epochs=40, lr=0.05 vs. epochs=150, lr=0.15). At 40 epochs neither configuration had converged, so the comparison couldn't distinguish 'dropout doesn't help' from 'neither network is done training yet.' At 150 epochs both configurations plateau, so the final numbers actually reflect each configuration's converged behavior rather than an arbitrary training-budget artifact.

**Q: Why does train_with_history call net.accuracy(), which itself calls net.forward(X, training=False), instead of reusing the training-mode probs already computed inside the epoch's mini-batch loop?**

A: Training-mode forward passes use dropout masks that randomly zero part of the network on each call -- they measure how well a randomly-thinned SUB-network fits, not the real, full network's accuracy. Evaluation must always run with training=False (mask=None) so dropout is fully off, matching exactly how the network will actually be used at inference time.

**Q: Why is eval_every=10 used instead of evaluating every single epoch?**

A: Evaluating requires a full forward pass over 200 train images and 80 test images -- roughly 1.4x the compute of one 200-image training epoch, done in one shot rather than split into batches. Doing that every single epoch across 150 epochs would meaningfully inflate runtime for resolution beyond what's actually needed to see the trend; every 10 epochs (15 data points across 150 epochs) is enough to see both the overfitting curve and the peak-test-accuracy epoch clearly.

**Q: The unregularized network's test accuracy is not monotonic -- it rises, peaks at epoch 40, dips, rises again, then plateaus low. Is that noise a bug?**

A: No -- mini-batch SGD with no momentum and a relatively high learning rate (0.15) naturally produces a noisy, non-monotonic trajectory, especially on only 200 training images. The important pattern is the ENVELOPE, not any single point: train accuracy's envelope rises steadily toward 1.0, while test accuracy's envelope never again reaches its early peak -- that's the overfitting signature, not any individual epoch's noise.

## Interview Preparation -- Theoretical Questions

**Q: What's the general principle for fairly comparing two regularization settings (or any two hyperparameter settings)?**

A: Both configurations must be trained to convergence -- or at least to the SAME degree of convergence -- before comparing final metrics. Comparing after a fixed, arbitrary number of steps risks measuring 'which one trains faster' rather than 'which one generalizes better,' which are different questions with potentially opposite answers, as Day 49 and Day 50 together demonstrate on the identical network.

**Q: Why does dropout capping train_acc at 0.8550 (rather than letting it reach 1.0 like the unregularized run) actually indicate the regularization is WORKING, not failing?**

A: Reaching train_acc=1.0 on a small, noisy dataset means the network has memorized every training example, including their noise -- not learned the general pattern. Dropout's randomly-thinned sub-network training makes that kind of exact memorization harder to achieve, which is precisely why it also generalizes better: the network is prevented from over-fitting to noise it would otherwise have the raw capacity to memorize.

**Q: If both dropout ON and dropout OFF were trained for only 20 more epochs past their observed test-accuracy peaks, would you expect the SIZE of their test-accuracy declines to be similar?**

A: No. The unregularized network, with nothing limiting its capacity to fit training-set-specific noise, is expected to keep drifting further from its peak the longer it trains past that point. The regularized network's dropout continues limiting how precisely it can fit any one training batch, which is consistent with today's observed result: its test accuracy holds at its peak through the end of training rather than declining further.

## Key Takeaway and What's Next

**Key takeaway:** A regularization comparison is only honest once both configurations have actually converged -- Day 49's identical network and dropout code, given only 40 epochs, could not distinguish 'dropout doesn't help here' from 'neither network finished training.' Given 150 epochs at a higher learning rate, the same exact code produces the textbook result: dropout OFF memorizes perfectly (train_acc=1.0000) but generalizes barely better than chance (test_acc=0.2875), peaking at epoch 40 and never recovering; dropout ON never fully memorizes (train_acc=0.8550) but reaches test_acc=0.7125, 2.5x better, and holds that gain through the end of training. train_with_history's epoch-interval tracking -- the one genuinely new piece of code today -- is what made the difference between an inconclusive snapshot and a real, honest trajectory visible.

Day 51 preview -- per the syllabus, Day 51 revisits L1/L2 weight regularization on CNNs, with a real over/underfitting demonstration. Today's fully-converged training setup (150 epochs, lr=0.15, train_with_history tracking) is the natural harness to reuse: same network, same honest convergence discipline, swapping dropout for weight decay to see whether it produces a similar, different, or complementary regularization signature.